

Feature Engineering in Machine Learning

Concepts, Techniques, and Practical Applications

Sourav Karmakar

Outline

- 1 What is Feature Engineering?
- 2 Overview of Techniques
- 3 EDA & Preprocessing
- 4 Feature Extraction
- 5 Feature Selection
- 6 Pipelines and Comparison
- 7 Practical Tips
- 8 Summary

What is Feature Engineering?

What is Feature Engineering?

Feature engineering is the process of transforming raw data into meaningful representations (*features*) that make machine learning models more effective.



It includes **cleaning**, **encoding**, **scaling**, **creating**, **selecting**, and **extracting**.

Why is Feature Engineering Important?

- **Better features > better algorithms**

Good features often matter more than choosing a fancy model.

- **Reduces overfitting**

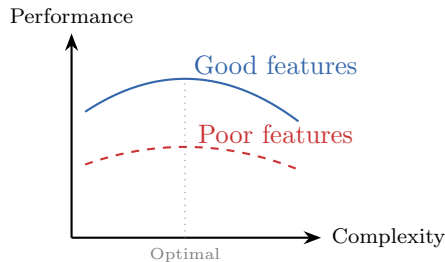
Removing noisy/irrelevant features improves generalization.

- **Simplifies models**

Fewer, better features speed up training and inference.

- **Improves interpretability**

Meaningful features make model decisions understandable.



Mathematical Formulation

Machine learning aims to approximate a function:

$$f^* : \mathbb{R}^d \rightarrow \mathbb{R} \quad (\text{or to a finite label set})$$

by learning parameters θ of a model $f(x; \theta)$.

Mathematical Formulation

Machine learning aims to approximate a function:

$$f^* : \mathbb{R}^d \rightarrow \mathbb{R} \quad (\text{or to a finite label set})$$

by learning parameters θ of a model $f(x; \theta)$.

Feature engineering transforms inputs via a mapping:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$$

so that the model

$$\hat{y} = f(\phi(x); \theta)$$

is **simpler**, **more regularized**, or **more accurate**.

Mathematical Formulation

Machine learning aims to approximate a function:

$$f^* : \mathbb{R}^d \rightarrow \mathbb{R} \quad (\text{or to a finite label set})$$

by learning parameters θ of a model $f(x; \theta)$.

Feature engineering transforms inputs via a mapping:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$$

so that the model

$$\hat{y} = f(\phi(x); \theta)$$

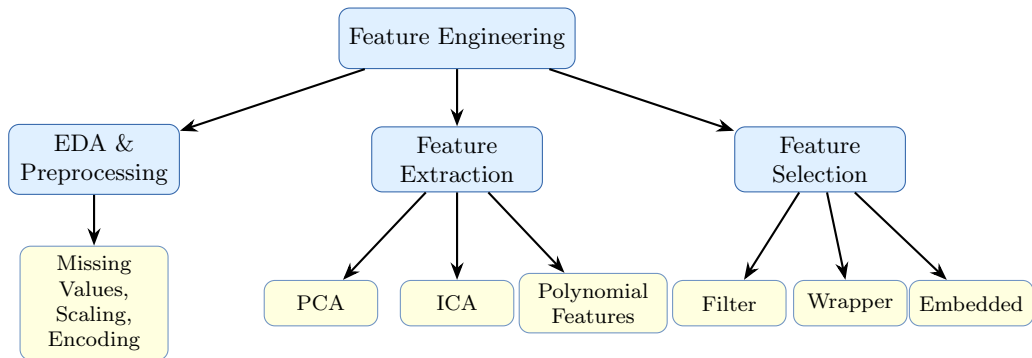
is **simpler**, **more regularized**, or **more accurate**.

Key Insight

The goal: find ϕ such that the downstream learning task becomes *easier*.

Overview of Feature Engineering Techniques

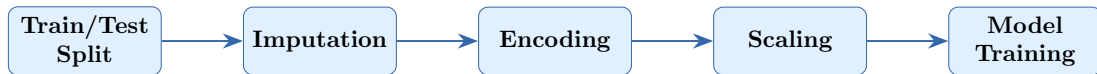
Taxonomy of Feature Engineering



EDA & Preprocessing

Preprocessing Pipeline

A typical preprocessing pipeline:



Golden Rule

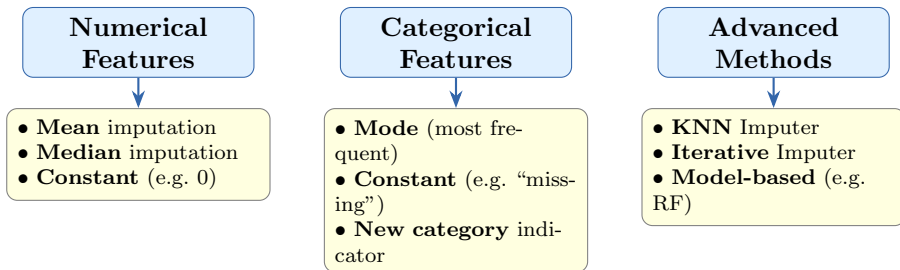
Always fit preprocessing steps on **training data only**, then transform both train and test. Violating this causes **data leakage**.

We now look at each step in detail: **Imputation**, **Encoding**, and **Scaling**.

Imputation: Handling Missing Values

Real-world datasets often have missing entries. **Imputation** fills them in.

The right strategy depends on the **type of data**:

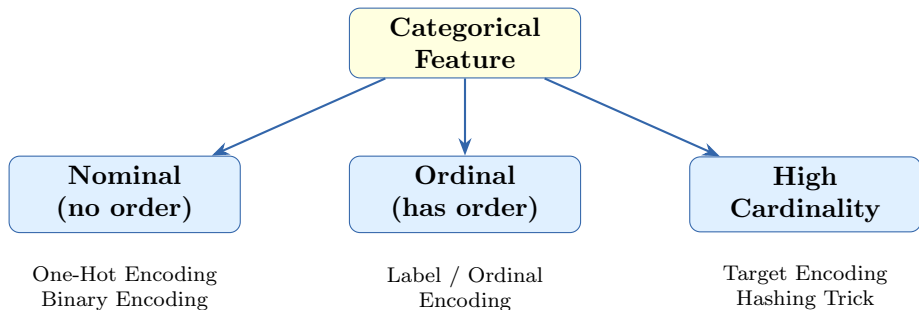


Imputation Strategies — When to Use What?

Strategy	Best For	Pros / Cons
Mean	Numerical, roughly symmetric	✓ Simple × Sensitive to outliers
Median	Numerical, skewed data	✓ Robust to outliers × Ignores feature correlations
Mode	Categorical	✓ Simple × Bias toward majority class
KNN Imputer	Any; correlated features	✓ Leverages neighbors × Expensive for large data
Iterative Imputer	Any; complex patterns	✓ Models inter-feature dependencies × Slow

Encoding Categorical Features — Overview

ML models require **numeric** inputs. Categorical features must be **encoded**.



One-Hot Encoding (Nominal Categories)

Each category c_k becomes a **binary indicator column**:

$$x_{\text{color}} \in \{\text{Red, Blue, Green}\} \longrightarrow [x_{\text{Red}} \quad x_{\text{Blue}} \quad x_{\text{Green}}]$$

Example:

Color	x_R	x_B	x_G
Red	1	0	0
Blue	0	1	0
Green	0	0	1

Pros:

- No artificial ordering
- Works with linear models

Cons:

- High cardinality $\Rightarrow$ high dimensionality
- Can cause *multicollinearity*

Tip: Drop-one trick

Use $K-1$ columns for K categories to avoid linear dependence (e.g. `drop='first'` in `sklearn`).

Label / Ordinal Encoding (Ordered Categories)

Assign an **integer** to each category, *preserving the natural order*:

$$x_{\text{size}} \in \{\text{S, M, L, XL}\} \longrightarrow x_{\text{size}} \in \{0, 1, 2, 3\}$$

Example:

Size (original)	Size (encoded)
S	0
M	1
L	2
XL	3

Warning

Never use label encoding for nominal (unordered) categories!

Assigning Red=0, Blue=1, Green=2 implies Red < Blue < Green, which misleads the model.

Target Encoding & Hashing (High Cardinality)

Target Encoding

Replace each category c_k with the *mean of the target*:

$$x_k \leftarrow \frac{1}{|S_k|} \sum_{i \in S_k} y_i$$

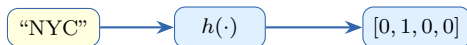
City	Encoded
NYC	0.72
LA	0.58
Chicago	0.65

× Risk of target leakage — use with cross-validation!

Feature Hashing

Apply a hash function to map categories into a *fixed-size vector*:

$$h(c_k) \bmod m \rightarrow \text{index in } \mathbb{R}^m$$



✓ Fixed dimensionality, no dictionary needed.

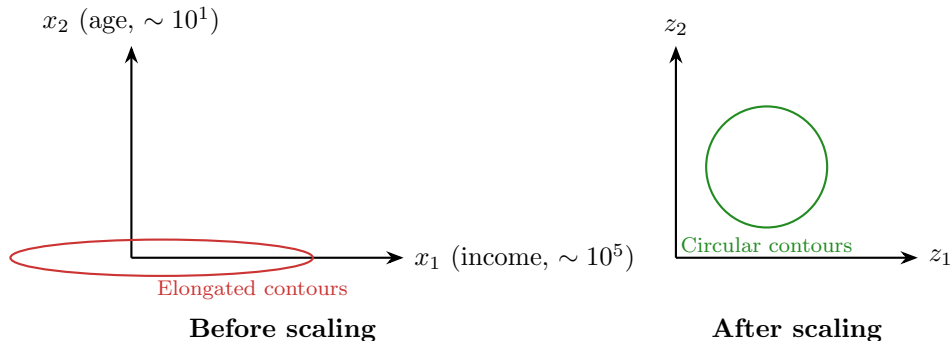
× Hash collisions may occur.

Encoding Summary

Technique	Category Type	Cardinality	Key Caveat
One-Hot	Nominal	Low–Medium	Dimensionality explosion
Label / Ordinal	Ordinal	Any	Must have true ordering
Target	Any	High	Target leakage risk
Feature Hashing	Nominal	Very High	Hash collisions
Binary	Nominal	Medium–High	Less sparse than One-Hot

Feature Scaling — Why?

Many models are sensitive to feature **magnitudes**:



Affected models: gradient descent (LR, NN), distance-based (KNN, SVM), PCA.

Unaffected models: tree-based methods (RF, XGBoost) are scale-invariant.

Standard Scaling (Z-score Normalization)

Given n training samples with feature values $x_1, x_2, \dots, x_n$ for a particular feature:

Standard Scaling (Z-score Normalization)

Given n training samples with feature values $x_1, x_2, \dots, x_n$ for a particular feature:

Step 1: Compute the **sample mean**: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$

Standard Scaling (Z-score Normalization)

Given n training samples with feature values $x_1, x_2, \dots, x_n$ for a particular feature:

Step 1: Compute the **sample mean**: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$

Step 2: Compute the **sample standard deviation**: $\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2}$

Standard Scaling (Z-score Normalization)

Given n training samples with feature values $x_1, x_2, \dots, x_n$ for a particular feature:

Step 1: Compute the **sample mean**: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$

Step 2: Compute the **sample standard deviation**: $\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2}$

Step 3: Transform each value:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Standard Scaling (Z-score Normalization)

Given n training samples with feature values $x_1, x_2, \dots, x_n$ for a particular feature:

Step 1: Compute the **sample mean**: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$

Step 2: Compute the **sample standard deviation**: $\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2}$

Step 3: Transform each value:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Properties

After transformation: $\frac{1}{n} \sum_i z_i = 0$ (zero mean), $\frac{1}{n-1} \sum_i z_i^2 = 1$ (unit variance).

Other Common Scalers

Scaler	Formula	When to Use
Min-Max	$z_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$	Need values in $[0, 1]$; sensitive to outliers
Standard	$z_i = \frac{x_i - \mu}{\sigma}$	Gaussian-like data; used with PCA/SVM
Robust	$z_i = \frac{x_i - \tilde{x}}{Q_3 - Q_1}$	Data with outliers; uses median $\tilde{x}$ and IQR

Notation

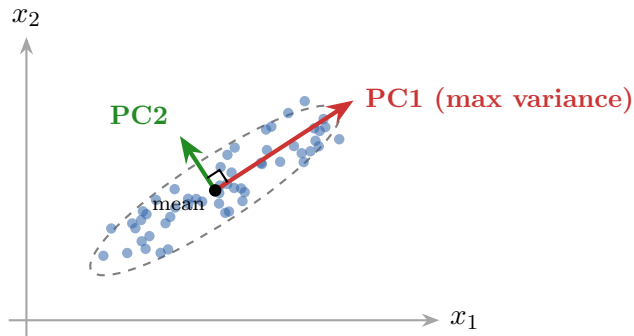
μ : sample mean; σ : sample standard deviation; $\tilde{x}$: median; Q_1, Q_3 : first and third quartiles; $\text{IQR} = Q_3 - Q_1$.

Feature Extraction

Creating new features from existing ones

Principal Component Analysis (PCA) — Intuition

Idea: Find orthogonal directions that maximize variance in the data.



PCA finds new axes (principal components) such that:

- PC1 captures the most variance
- PC2 captures the next most (**orthogonal** to PC1)
- And so on...

PCA — Mathematical Details

Given centered data matrix $X \in \mathbb{R}^{n \times d}$:

PCA — Mathematical Details

Given centered data matrix $X \in \mathbb{R}^{n \times d}$:

Step 1: Compute covariance matrix:

$$C = \frac{1}{n-1} X^\top X \in \mathbb{R}^{d \times d}$$

PCA — Mathematical Details

Given centered data matrix $X \in \mathbb{R}^{n \times d}$:

Step 1: Compute covariance matrix:

$$C = \frac{1}{n-1} X^\top X \in \mathbb{R}^{d \times d}$$

Step 2: Eigendecomposition:

$$C = V \Lambda V^\top$$

where $V = [v_1, v_2, \dots, v_d]$ are eigenvectors and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ with $\lambda_1 \geq \lambda_2 \geq \dots$.

PCA — Mathematical Details

Given centered data matrix $X \in \mathbb{R}^{n \times d}$:

Step 1: Compute covariance matrix:

$$C = \frac{1}{n-1} X^\top X \in \mathbb{R}^{d \times d}$$

Step 2: Eigendecomposition:

$$C = V \Lambda V^\top$$

where $V = [v_1, v_2, \dots, v_d]$ are eigenvectors and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ with $\lambda_1 \geq \lambda_2 \geq \dots$.

Step 3: Project onto top k components:

$$Z = X \cdot V_k \in \mathbb{R}^{n \times k}$$

PCA — Mathematical Details

Given centered data matrix $X \in \mathbb{R}^{n \times d}$:

Step 1: Compute covariance matrix:

$$C = \frac{1}{n-1} X^\top X \in \mathbb{R}^{d \times d}$$

Step 2: Eigendecomposition:

$$C = V \Lambda V^\top$$

where $V = [v_1, v_2, \dots, v_d]$ are eigenvectors and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ with $\lambda_1 \geq \lambda_2 \geq \dots$.

Step 3: Project onto top k components:

$$Z = X \cdot V_k \in \mathbb{R}^{n \times k}$$

Variance explained by k components: $\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$

Independent Component Analysis (ICA) — Intuition

Idea: Observed data X is a *linear mixture* of latent independent sources S :

$$X = A \cdot S$$

Independent Component Analysis (ICA) — Intuition

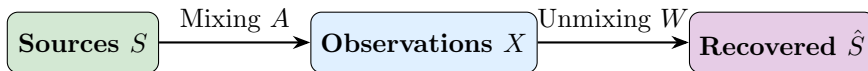
Idea: Observed data X is a *linear mixture* of latent independent sources S :

$$X = A \cdot S$$

ICA recovers S by estimating an *unmixing matrix* W :

$$S \approx W \cdot X$$

with the constraint that components of S are **statistically independent**.



Why Non-Gaussianity Matters in ICA

Central Limit Theorem (CLT):

- Adding independent random variables $\Rightarrow$ result tends toward Gaussian.
- Therefore: **mixtures** of signals $\rightarrow$ *more Gaussian*.
- Individual independent sources $\rightarrow$ *less Gaussian* (more “non-Gaussian”).

ICA Strategy

Maximize **non-Gaussianity** (e.g., kurtosis or negentropy) of the recovered signals to find the independent components.

PCA vs. ICA

- **PCA:** finds uncorrelated components maximizing variance
- **ICA:** finds *independent* components maximizing non-Gaussianity

Polynomial and Interaction Features

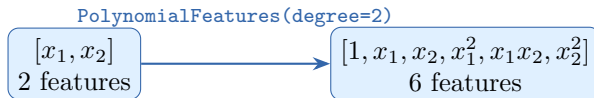
Idea: Map inputs to higher dimensions including interactions.

For degree 2, the mapping ϕ transforms:

$$\phi(x) = [1, x_1, \dots, x_d, x_1^2, \dots, x_d^2, x_1x_2, \dots]$$

Example: If $x = [x_1, x_2]$, degree 2 gives:

$$\phi(x) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$$



Caution: Feature count grows combinatorially! $\binom{d+k}{k}$ for degree k .

Feature Selection

Choosing a subset of existing features

Feature Selection: Three Approaches

Independent of model

Filter Methods

Variance Threshold
F-test (ANOVA)
Mutual Information

Trains a model

Wrapper Methods

RFE
RFECV

Built into model

Embedded Methods

L1 (LASSO)
Tree-based
Importance

Increasing computational cost →

Filter Method: Variance Threshold

Idea: Remove features with low variance (near-constant features add no information).

A feature with variance $\sigma^2 = 0$ is constant $\Rightarrow$ useless!

Example:

Feature	Variance	Keep?
x_1	12.5	✓
x_2	0.01	✗
x_3	8.3	✓
x_4	0.0	✗

With threshold = 0.5, features x_2 and x_4 are removed.

Filter Method: Univariate F-test (ANOVA) I

ANOVA compares variance *between groups* vs. variance *within groups*.

For a feature x , the F-statistic is:

$$F = \frac{\text{variance between groups}}{\text{variance within groups}}$$

High F-score $\Rightarrow$ feature is *more discriminative* for the target.



F-test — Numerical Example

Data: Feature x measured across two classes.

Class 0:	$x = \{1, 2, 3\}$	$\bar{x}_0 = 2$
Class 1:	$x = \{7, 8, 9\}$	$\bar{x}_1 = 8$
Overall:	$\bar{x} = 5, \quad n_0 = n_1 = 3, \quad K = 2$	

Step 1: Between-group variance (how far group means are from the overall mean):

$$SS_{\text{between}} = \sum_k n_k (\bar{x}_k - \bar{x})^2 = 3(2 - 5)^2 + 3(8 - 5)^2 = 27 + 27 = 54$$

Step 2: Within-group variance (how far points are from their own group mean):

$$SS_{\text{within}} = \sum_k \sum_{i \in k} (x_i - \bar{x}_k)^2 = (1+0+1) + (1+0+1) = 4$$

Step 3: F-statistic:

$$F = \frac{SS_{\text{between}}/(K - 1)}{SS_{\text{within}}/(N - K)} = \frac{54/1}{4/4} = \frac{54}{1} = \boxed{54}$$

Interpretation

$F = 54 \gg 1$ — the group means are very far apart relative to the spread within each group. This feature is **highly discriminative**.

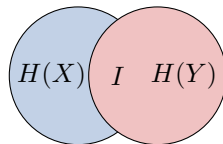
Filter Method: Mutual Information (MI) I

Measures *general dependency* between feature X and target Y :

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x) p(y)}$$

Properties:

- $I(X; Y) \geq 0$
- $I(X; Y) = 0 \Leftrightarrow X$ and Y are independent
- Captures **both linear and non-linear** dependencies



$$I(X; Y) = H(X) + H(Y) - H(X, Y)$$

MI vs. F-test

F-test only captures linear relationships; MI captures any dependency.

Mutual Information — Numerical Example

Data: Binary feature X and binary target Y over 100 samples.

	$Y = 0$	$Y = 1$	Total
$X = 0$	40	10	50
$X = 1$	10	40	50
Total	50	50	100

Marginals: $p(X=0) = p(X=1) = 0.5$, $p(Y=0) = p(Y=1) = 0.5$.

Joint probabilities: $p(0,0) = 0.4$, $p(0,1) = 0.1$, $p(1,0) = 0.1$, $p(1,1) = 0.4$.

Compute MI:

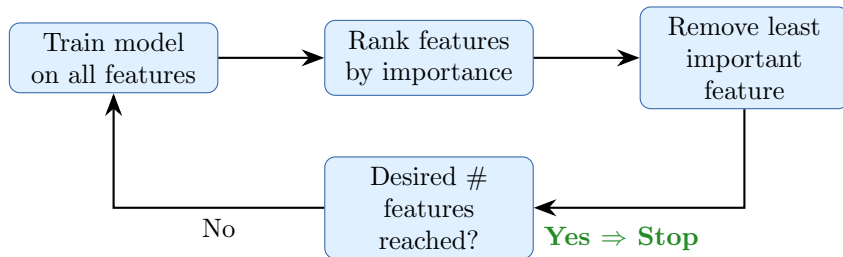
$$\begin{aligned} I(X; Y) &= \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)} \\ &= 2 \times 0.4 \log_2 \frac{0.4}{0.25} + 2 \times 0.1 \log_2 \frac{0.1}{0.25} = 0.8 \log_2 1.6 + 0.2 \log_2 0.4 \\ &= 0.8(0.678) + 0.2(-1.322) = 0.542 - 0.264 = \boxed{0.278 \text{ bits}} \end{aligned}$$

Interpretation

$I > 0$ confirms X and Y are **dependent**. Knowing X gives ~ 0.28 bits of information about Y .

Wrapper Method: Recursive Feature Elimination (RFE)

Idea: Iteratively remove the *least important* feature based on a model.



RFECV: Uses cross-validation to automatically choose the optimal number of features.

Embedded Method: L1 Regularization (LASSO)

Idea: L1 penalty encourages *sparsity* in model coefficients.

Embedded Method: L1 Regularization (LASSO)

Idea: L1 penalty encourages *sparsity* in model coefficients.

LASSO objective:

$$\min_{\beta} \underbrace{\frac{1}{2n} \|y - X\beta\|_2^2}_{\text{Loss}} + \underbrace{\lambda \|\beta\|_1}_{\text{L1 penalty}}$$

Embedded Method: L1 Regularization (LASSO)

Idea: L1 penalty encourages *sparsity* in model coefficients.

LASSO objective:

$$\min_{\beta} \underbrace{\frac{1}{2n} \|y - X\beta\|_2^2}_{\text{Loss}} + \underbrace{\lambda \|\beta\|_1}_{\text{L1 penalty}}$$

The $\|\beta\|_1 = \sum_j |\beta_j|$ penalty *drives some coefficients exactly to zero*, effectively performing feature selection.

The L1 “diamond” tends to hit axes $\Rightarrow$
sparse solutions.

Embedded Method: L1 Regularization (LASSO)

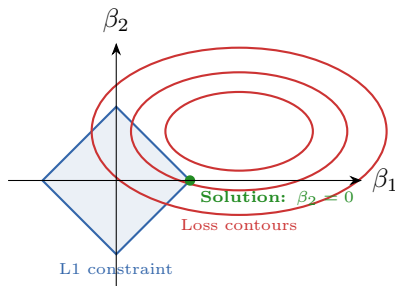
Idea: L1 penalty encourages *sparsity* in model coefficients.

LASSO objective:

$$\min_{\beta} \underbrace{\frac{1}{2n} \|y - X\beta\|_2^2}_{\text{Loss}} + \underbrace{\lambda \|\beta\|_1}_{\text{L1 penalty}}$$

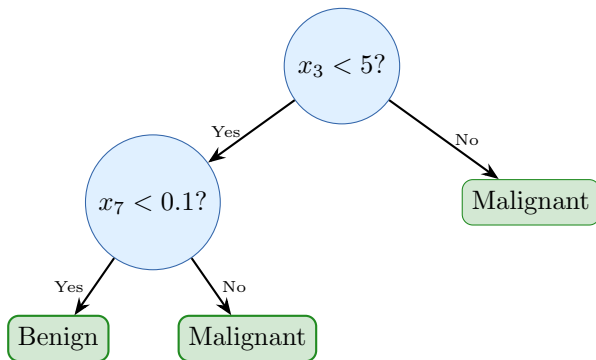
The $\|\beta\|_1 = \sum_j |\beta_j|$ penalty *drives some coefficients exactly to zero*, effectively performing feature selection.

The L1 “diamond” tends to hit axes $\Rightarrow$ **sparse solutions.**



Embedded Method: Tree-based Feature Importance

Idea: Decision trees split on the most informative features. The total impurity reduction from each feature measures its importance.



Random Forests aggregate importance across *many trees*, yielding robust feature rankings.

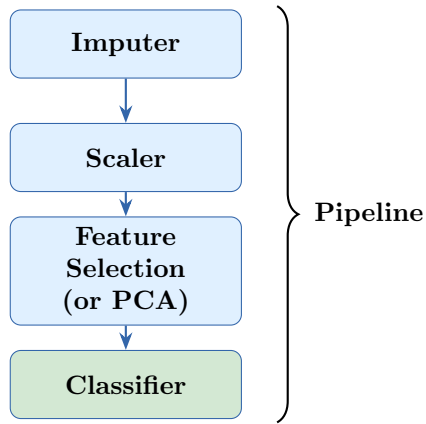
Pipelines and Comparison

Why Use Pipelines?

Pipelines chain preprocessing and modeling steps into a single object.

Benefits:

- **Prevents data leakage:** scaler is fit only on training folds during cross-validation.
- **Reproducibility:** all steps are encapsulated.
- **Convenience:** single `.fit()` and `.predict()` call.



Experimental Comparison (Breast Cancer Dataset)

Setup: 5-fold cross-validation with Logistic Regression.

Pipeline	Mean Accuracy	Std
Baseline (all 30 features)	0.981	0.007
SelectKBest (F-test, $k=10$)	0.951	0.026
PCA (10 components)	0.981	0.007
ICA (10 components)	~ 0.986	—
Poly Features (degree=2)	~ 0.965	—

Observation

Feature extraction (PCA, ICA) can match or exceed the baseline accuracy with **fewer features**, while naive selection (SelectKBest) may drop useful information.

Practical Tips

Best Practices for Feature Engineering

- ➊ **Always split train/test *before* feature selection/extraction.** Prevents data leakage and gives honest performance estimates.
- ➋ **Use pipelines** to ensure transformations are consistently applied.
- ➌ **Leverage domain knowledge** when possible. Expert-crafted features often outperform automated approaches.
- ➍ **For high-dimensional data**, start with filters or L1 models to quickly reduce dimensionality.
- ➎ **Combine methods:** e.g., filter out low-variance features first, then apply PCA or wrapper methods on the remainder.
- ➏ **Validate rigorously:** use cross-validation to compare different feature engineering strategies.

Common Pitfalls

Data Leakage

Fitting scaler or selector on *full* data before splitting $\Rightarrow$ overly optimistic results.

Ignoring Feature Correlations

Highly correlated features can destabilize models (e.g., linear regression). Consider removing redundant features.

Curse of Dimensionality

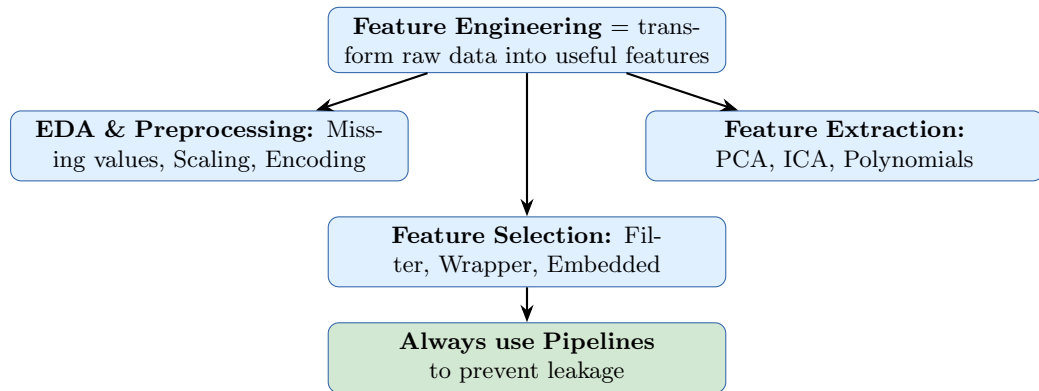
Too many features (e.g., degree-2 polynomials on 30 features $\rightarrow$ 496 features) can hurt performance.

Over-engineering

Adding too many hand-crafted features can lead to overfitting. Always validate!

Summary

Summary of Key Concepts



Key Takeaway

Good feature engineering is often **more impactful** than choosing a complex model. Invest time in understanding and transforming your data!

Thank You!

Questions?